SWE 4404: SOFTWARE PROJECT LAB II

Summer 2024-25

LazyCow.

An Automated Desktop Workspace Orchestrator

Tamjidur Rahman
230042109

Shudipto Sarwar
230042142

Irfan Sadik Chy
230042147

Mohammad Faiaj Jarif
230042152

Department of Computer Science and Engineering (CSE)
Islamic University of Technology

The Problem & Solution

The Problem: Fragmented Friction

- **Manual Initialization:** Opening IDEs, terminals, and browser tabs sequentially wastes 10-15 minutes daily.
- **Fragmented Execution:** Resizing windows and launching scripts across disconnected OS utilities.
- **No Cross-Device Sync:** Layouts and file paths are tethered to a single machine.

The Solution: Unified Automation

- **Hotkey Orchestration:** Unifies every launch into a single keystroke via Electron.
- **Window Layout Engine:** Snaps apps into 7 predefined zones using native OS calls.
- **Visual Builder:** Drag-and-drop sequencing of 10 action types.
- **Cloud Sync:** Supabase integration with path sanitization for portability.

Technology Stack

A modern, type-safe ecosystem bridging web technologies with native Windows APIs.

Frontend UI

- **React 18 & TypeScript:** Robust component architecture.
- **Tailwind CSS v3:** Utility-first styling.
- **Vite 5:** Ultra-fast bundler and dev server.
- **@dnd-kit & Motion:** Drag-and-drop and fluid animations.

Desktop Backend

- **Electron (v30):** Desktop shell & Node.js runtime.
- **Node.js:** Handles IPC and file system ops.
- **C# P/Invoke:** Direct Win32 API window snapping.
- **PowerShell:** Secure OS automation execution.

Cloud & Database

- **Supabase:** Open-source backend platform.
- **PostgreSQL:** Relational database profiles.
- **Row-Level Security:** Strict data isolation.
- **Google OAuth:** Secure authentication via Deep Links.

Project Structure

Strict module boundaries isolate privileged OS execution from the stateless React renderer.

```
LazyCow/
├── electron/
│   ├── main.ts         # App lifecycle, IPC handlers, OS execution
│   ├── preload.ts      # Whitelist-enforced contextBridge
│   ├── windowLayout.ts # Native window snapping engine
│   └── preferences.ts  # Hardware acceleration config
├── src/
│   ├── components/     # React UI (Builder, Library, Settings)
│   ├── hooks/          # Custom logic (useHotkeyRecorder, useDragSelect)
│   ├── pages/          # App routing views
│   ├── types/          # Shared TypeScript DTOs (actions.ts)
│   └── utils/          # Cloud sync, disk mirror, safety classifiers
├── index.html          # Splash screen & CSP configuration
└── package.json        # Dependencies & build scripts
```

Component Communication

The Data Class (DTO)

```
interface SavedShortcut {
  id: string;
  name: string;
  hotkey: string;
  actions: ActionItem[];
  windowLayout: WindowLayoutConfig;
}
```

Immutable Schema Contract

Type-safe DTOs traverse a secure bridge through four execution phases.

Shared TypeScript DTOs establish an immutable schema contract between frontend and backend, preventing desynchronization across the IPC bridge.

Communication Flow

1. Trigger
User fires hotkey or clicks Run.

→

2. Transport
DTO dispatched via contextBridge.

→

3. Validation
Node.js pre-flights paths & Auth.

→

4. Execution
OS APIs launch apps & snap windows.

End-to-End Feature Request

Example: Executing a Shortcut via Global Hotkey

Frontend

- User presses global hotkey (e.g., `Ctrl+Shift+W`).
- Electron intercepts the OS signal.
- React UI updates progress ring.

Backend

- Node.js iterates action array.
- Triggers native OS shell to launch apps.
- Invokes Layout Engine for snapping.

Database

- Cloud Sync calculates diffs.
- Pushes JSONB to PostgreSQL.
- RLS ensures secure data ownership.

Individual Contributions

Irfan (Builder & UI)

Built the entire Visual Builder: drag-and-drop sequencing, hotkey recording, path validation, and local disk restore mechanism.

Jarif (OS Engine)

Built backend execution: native IPC communication, the C# window snapping engine, and the full Window Layout UI panel.

Shudipto (Library & Shell)

Built dashboard and navigation: Library view, IPC bridge, app routing, and all desktop Settings configurations.

Tamjid (Cloud & Sync)

Built Supabase integration: authentication, cloud sync engine, path sanitization hints, and the App Drawer.

Thank You

We are open for questions and code demonstrations.

GitHub Repository: LazyCow